



PROJECT AND TEAM INFORMATION

Project Title

Vaultify – A Secure Command-Line Password Manager in C++

<i>Team Id:</i>	<i>DSCPP-III-2026-T305</i>
Team member 1 (Team Lead) 	<i>Mishra, Ashraya</i> <i>Student ID: 2510010429</i> <i>Roll No.: 2027681</i> <i>Section : B</i>
Team member 2 	<i>Joshi, Udit</i> <i>Student ID: 2510370397</i> <i>Roll No.: 2028891</i> <i>Section : ML2</i>
Team member 3 	<i>Singh, Yuvraj</i> <i>Student ID: 2510010591</i> <i>Roll No.: 2028915</i> <i>Section: ML3</i>

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

Vaultify is a secure, offline, command-line password manager built in C++17 for a single user on a single machine. It addresses the widespread problem of password reuse and weak passwords by letting a user store strong, unique credentials for every account while remembering only one master password. The project is built from first principles rather than relying on existing libraries: credentials are stored in a manually implemented hash table (using separate chaining for collision resolution) that gives $O(1)$ average lookup by site name; stored passwords are protected using a symmetric XOR cipher keyed on the master password; and the master password itself is never stored anywhere – only a one-way hash of it is kept on disk and used purely for login verification. A dedicated `FileHandler` module persists the encrypted vault to a local flat file so data survives across sessions, and a menu-driven CLI ties every module together. The project deliberately applies core Data Structures concepts (hashing, collision resolution, stacks) and Object-Oriented Programming concepts (encapsulation, class design, file handling) taught in this course, rather than using built-in STL containers, so that the underlying mechanics are demonstrated rather than abstracted away.

(Describe your current approach, including system design, communication protocols, libraries used, etc.

Max 300 words).

The system is organized into six independent modules, each matching a single responsibility: **Entry** (the credential data class – site, username, encrypted password, fully encapsulated), **Vault** (the hash table storage engine using separate chaining, with dynamic resizing once the load factor exceeds roughly 0.7), **Crypto** (the XOR symmetric cipher for encrypting/decrypting stored passwords, plus a custom djb2-style hash function used only to verify the master password at login – never to decrypt data), **FileHandler** (serializes the vault to a pipe-delimited flat file and reconstructs it on startup), **RecentStack** (a LIFO stack tracking the last few retrieved sites), and **CLI** (the menu-driven interface that calls into the other modules but contains no business logic itself). This separation means the CLI could later be replaced with a GUI without touching any of the underlying logic. No external libraries or frameworks are used – only the C++ standard library (`<string>`, `<vector>`, `<fstream>`, `<random>`) – since the goal is to demonstrate the data structures and OOP concepts ourselves rather than depend on pre-built abstractions such as `std::unordered_map`. The tool is single-process and local-only, so there is no network communication or client-server protocol involved; all "communication" is between in-process modules via well-defined class interfaces. Development uses Visual Studio Code with GCC/g++ (C++17), Git for version control, and the project compiles cleanly on both Windows (MinGW) and Linux with no platform-specific code paths.

Tasks Completed (7 pts)

Task Completed	Team Member
Vault module – hash table with separate chaining, insert/retrieve/update/delete/list operations	Ashraya Mishra
FileHandler module – vault persistence (save/load) to a local encrypted flat file	Ashraya Mishra
Entry class – encapsulated credential data model	Udit Joshi
Crypto module – XOR cipher for password encryption/decryption and djb2-based master password hashing	Udit Joshi
CLI module – menu-driven interface, input validation, colored/formatted output	Yuvraj Singh
RecentStack module and initial cross-platform (Windows/Linux) build testing	Yuvraj Singh

Challenges/Roadblocks (7 pts)

One early challenge was deciding how to resolve hash collisions in the Vault without relying on `std::unordered_map` – we evaluated open addressing against separate chaining and chose separate chaining, since it handles a higher load factor more gracefully and avoids the extra complexity of tombstone management needed when deleting entries under open addressing. A second challenge was a subtle design risk in Crypto: an early draft risked using the stored master-password hash itself as the decryption key, which would have meant anyone who stole the vault file could decrypt every entry without ever knowing the real password. We corrected this so the hash is used *only* for login verification, while the actual decryption key is always the plain-text master password typed by the user in that session, never persisted anywhere. A third, ongoing challenge is keeping the hash table's performance genuinely $O(1)$ as more entries are added – we are implementing dynamic resizing (doubling the table and rehashing once load factor crosses roughly 0.7) to prevent long chains from degrading lookup speed. Finally, ensuring identical behavior on both Windows (MinGW) and Linux required care around file path handling and verifying that ANSI color codes render correctly across terminals; we are testing on both platforms in parallel rather than treating cross-platform support as an afterthought at the end.

Tasks Pending

Task Pending	Team Member (to complete the task)
Dynamic hash table resizing when load factor exceeds threshold	Ashraya Mishra
Random password generator and password-strength checker (stretch goals)	Udit Joshi
Full regression testing across both platforms, final CLI polish, and README finalization	Yuvraj Singh

Project Outcome/Deliverables

The completed project will deliver: a fully functional, cross-platform C++ command-line password manager compiled and tested on both Windows and Linux; complete, well-commented source code implementing the Entry, Vault, Crypto, FileHandler, RecentStack, and CLI modules; a compiled executable ready for direct use; and a final report (with this progress document and the Phase-I proposal) documenting the design, security model, and stated limitations of the system. The core deliverable – secure local storage and retrieval of credentials without ever storing the master password in plain text – is already functioning end-to-end in the current build.

Progress Overview

All five core goals from the Phase-I proposal are functionally in place: the hash table vault, XOR-based encryption/decryption, hash-based master password authentication, file persistence, and a working menu-driven CLI. This puts the project on schedule against the TA2 milestone defined in Phase-I. Ahead of schedule: the RecentStack feature (originally a stretch goal) and initial CLI output polish (colored, formatted output) are already implemented. Slightly behind schedule: dynamic hash table resizing is designed but not yet fully implemented, and the two remaining stretch goals (password generator, strength checker) have not been started. Both are scoped to be completed well before the TA3 final demonstration, and neither blocks the core, already-working functionality.

Codebase Information

Repository: <https://github.com/axhraya1121/password-manager> . Primary branch: main. Development follows a modular commit history aligned to the module ownership above – key commits include the initial project scaffold (Entry, Vault, Crypto, FileHandler, CLI headers and build setup), the working Vault hash table with separate chaining, the Crypto module implementing the XOR cipher and djb2 master-password hashing with the hash-vs-encryption-key separation, the FileHandler persistence layer, and the CLI integration commit tying all modules together into a runnable end-to-end build. All members commit directly to main with regular small commits per completed sub-task, following the module boundaries defined in the Phase-I Project Approach.

Testing and Validation Status

Test Type	Status (Pass/Fail)	Notes
Vault insert/retrieve/update/delete	Pass	Verified O(1) average lookup behavior manually across varying entry counts
Encryption/decryption round-trip	Pass	Encrypted then decrypted passwords match original input exactly
Master password authentication	Pass	Correct password grants access; incorrect password rejected after 3 attempts
File persistence across sessions	Pass	Vault contents correctly reloaded after closing and relaunching the program
Cross-platform build	Pass	Compiled and run successfully on both Windows (MinGW) and Linux

Deliverables Progress

- **Completed:** Entry class, Vault (hash table with separate chaining), Crypto (XOR cipher + master password hashing), FileHandler (persistence), RecentStack, and a working menu-driven CLI with colored output.
- **In progress:** Dynamic hash table resizing; final cross-platform regression testing.
- **Pending:** Random password generator and password-strength checker (stretch goals); final report and polished README.